Attention

How tokens share meaning

LLM chapter, session 3. Parts adapt LMU I2DL (CC BY 4.0) and 3Blue1Brown, “Deep Learning” chapters 5 and 6.

Before we start: two videos and a demo

Recommended viewing

3Blue1Brown, Attention in transformers, step-by-step | Deep Learning Chapter 6 (2024, 26 min): today's lecture, animated. Many stills in this deck are from it.

Andrej Karpathy, Let's build GPT: from scratch, in code, spelled out. (2023). From 42:13: attention built in code, one version at a time; why divide by $\sqrt{d_k}$ at 1:16:56.

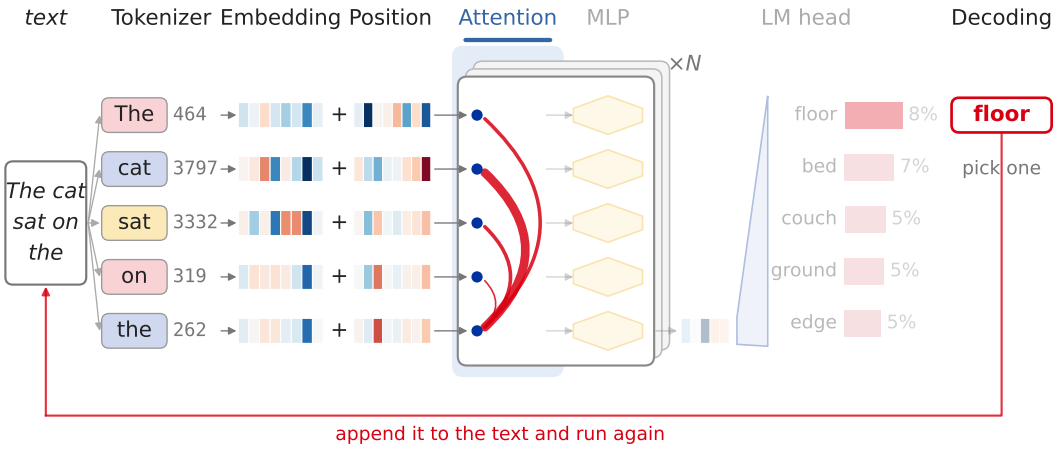
Live demo: Transformer Explainer

GPT-2 small running in your browser (about a 600 MB download): type a sentence, open one of the 12 heads, watch $QK^\top$, softmax and the weights.

Try:

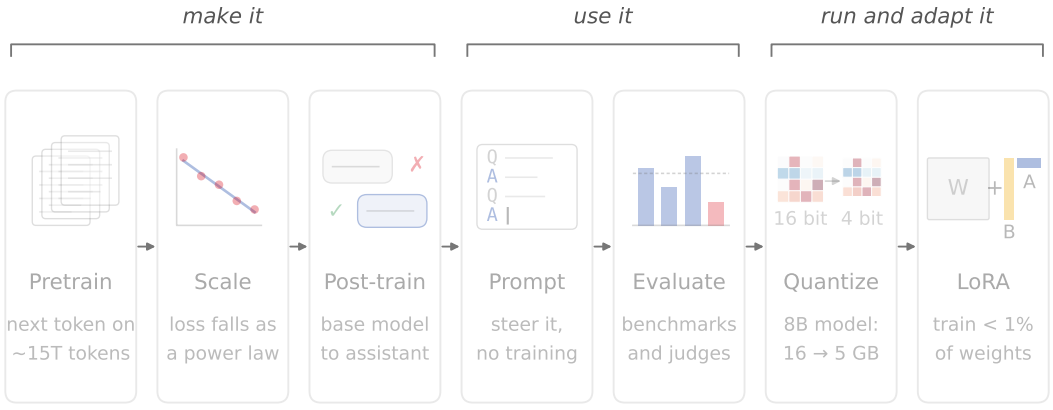
- our creature sentence: does a head send creature to fluffy?
- a sentence with "it": which head finds what "it" is?
- the same head on two different sentences

Where we are: one forward pass



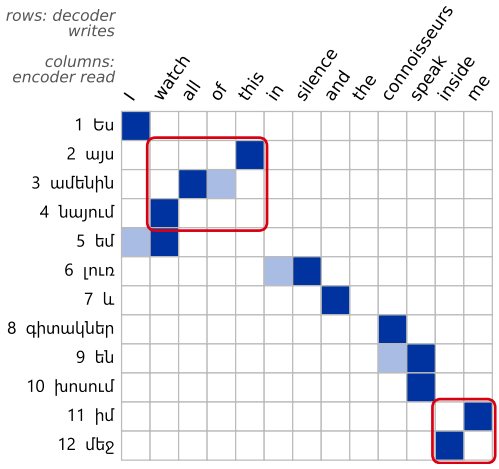
Today: the **attention** step - how each token's vector picks up meaning from the other tokens.

Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

[54] left two walls



Wall 1: one C for every step. The decoder writes the Armenian line one word per row. Each row needs one or two *different* English words, and seq2seq hands every row the same context vector C.

Wall 2: the queue. The RNN reads one token after another: 1,000 tokens are 1,000 steps in a row, while most of the GPU waits.

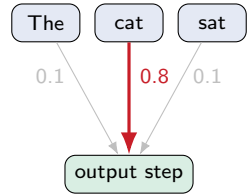
Predict: how would you fix wall 1? What should row 4 ("watch") get instead of C?

The fix, in one idea

Don't force everything through one vector. At **each** output step, let the decoder **look back over every input state** and take a *weighted average*: heavy weight on the words this step needs, light weight on the rest.

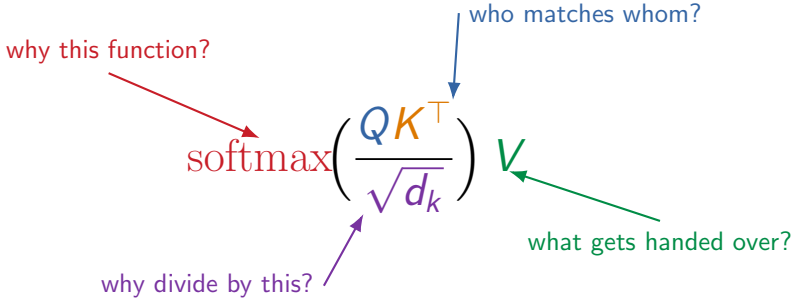
Row 4 ("watch") puts most of its weight on "watch"; row 5 picks a different mix. Every step gets its own context.

That weighted look-back *is* **attention**. Today unpacks this one sentence, and then takes it much further.



weights sum to 1; this step leans on "cat"

Where we are going: one line



This line is the whole of today. By the end, every piece of it has a job you can explain. Walls 1 and 2 say why the line exists at all; then we take it apart one piece at a time, and finally see what the model may *not* look at: the mask.

Outline

Wall 1: look back

Wall 2: drop the queue

One head, slowly

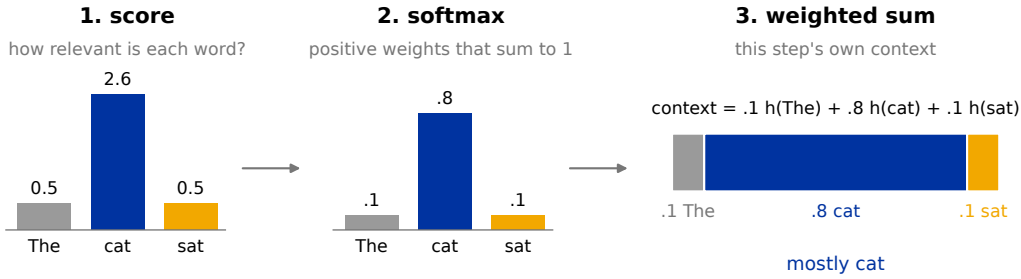
Masking: who may look at whom

Wall 1: look back

A different context for every word you write.

Score, softmax, weighted sum

At each output step, attention does three things. The step from the fix, as a picture:



The blend is this step's own context, its personal C. The decoder's current state does the asking, the encoder's states answer. The score itself: Bahdanau et al. fed the two states to a small learned network; later models just take their dot product, as we will.

You have seen the three moves: LLM-3 ran them by hand on "dog bites man", and softmax and weighted sums are [41]. The new part is *what* gets scored and blended.

Learned, not drawn

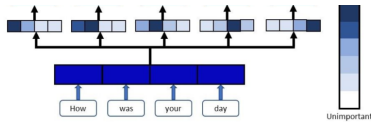


Figure: An illustration of a machine translation task using an encoder-decoder model with an attention mechanism. The attention weights at each time-step of the decoding/translation process indicate which parts of the input sequence are most relevant. There are 4 attention weights because there are 4 encoder states (Loye, 2019).



A woman is throwing a frisbee in a park.



A stop sign is on a road with a mountain in the background.

Translation: at each output step the model learns its own weights over the input words (the bar on the right).

(Bahdanau, Cho & Bengio, 2015: the first attention)

Captioning: weights over *pixels* while writing each word. (Xu et al., 2015)

Stack the steps and you get a grid like [54]'s, drawn by training, not by hand: nobody labels the alignments, only the translation loss trains them. Wall 1 is down.

Source: LMU I2DL (slds-lmu), CC BY 4.0

Wall 2: drop the queue

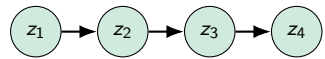
If every word can look at every word, who needs the recurrence?

The encoder still reads one word at a time

Attention took down wall 1: every output step gets its own mix of the encoder's states.

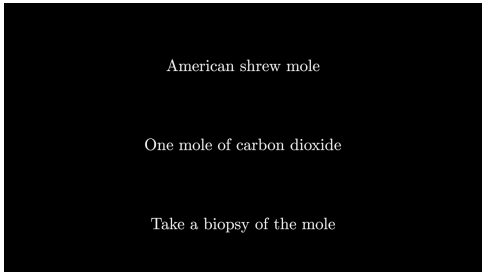
But those states are still built by an RNN, one after another: z_2 waits for z_1, z_3 waits for z_2. Wall 2 is still standing.

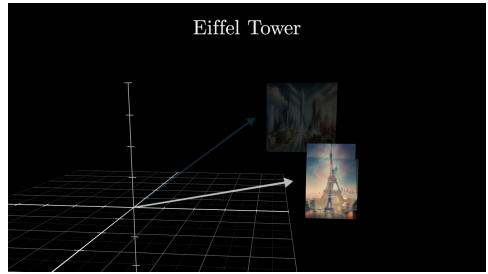
Predict: attention lets any position read any other position directly. Could it build the vectors themselves, with no recurrence at all?



each one waits for the one before

Words need their own sentence





"mole" means three different things, but the vector it starts with - its row of the table (LLM-3) - is identical in all three.

"tower", after "Eiffel", should move toward Paris, France, steel.

Same with "a machine learning **model**" and "a fashion **model**". Each word's vector has to change with its sentence, and what it needs is in that same sentence.

3Blue1Brown

"Each of you has what on the surface appears an unassailable alibi." she said, as the storm cleared and the crimson rays of dusk poured through the window where everyone was gathered. "But only one of you could have known about the loose screw on the window, while also having known where the second key was hidden. I am left, inescapably then, to the conclusion that therefore, the murderer was ???



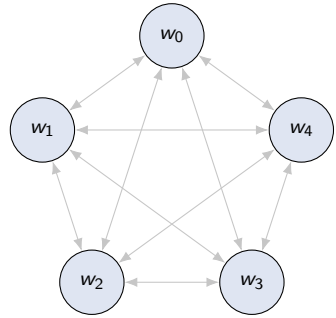
To fill the blank, the last vector must have soaked up the whole story.

The sentence talks to itself

In translation the *decoder's* state did the asking and the *encoder's* states answered: two sequences.

In **self**-attention the asking and the answering both come from the **same** sentence. Every word asks, and every word answers. One round is all-to-all messaging: each word gathers what it needs from all the others.

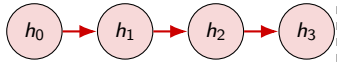
So yes to the predict: each word's new vector is built from all the others in one step, and the recurrence goes. The 2017 paper said it in its title: **"Attention Is All You Need."** (Vaswani et al., 2017)



every word attends to every word

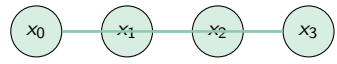
Sequential vs. all-to-all

RNN / LSTM



word 1 reaches word n after n steps

Self-attention



all-to-all: one step, direct

Three wins: **parallel** (all tokens at once) · **direct** (any token to any other) · **contextual** (each vector changes with its sentence). And it is all matrix products: the work a GPU is built for ([40], [54]'s second wall).

And word order? Self-attention alone cannot see it (LLM-3's "dog bites man"). That is why the position information goes into the vectors *before* the first attention layer.

The paper behind the title

- Vaswani et al. (2017): eight authors at Google, and a footnote that reads "Equal contribution. Listing order is random." Its main results were in machine translation.
- The title was Llion Jones's idea, after the Beatles' "All You Need Is Love". The name "Transformer" was there from day zero; Jakob Uszkoreit had two Transformer toys as a kid.
- All eight later left Google and founded or joined companies: Cohere, Character.AI, Sakana AI, among others. One came back: Noam Shazeer returned in 2024.
- Google Scholar today: about **279,000** citations. Within a year the block powered GPT and BERT (LLM-6).

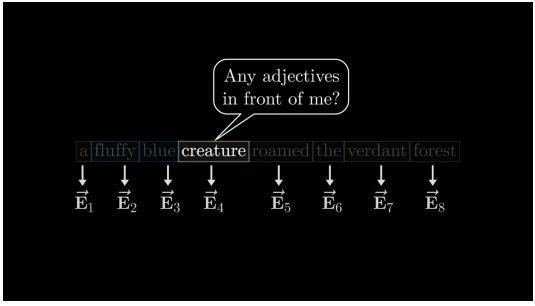
A translation paper that set out to drop the recurrence. Attention is the part that stayed.

Wired (Levy, 20 March 2024), "8 Google Employees Invented Modern AI. Here's the Inside Story"; the paper's footnote; Google Scholar, read 27 Oct 2026 (278,670).

One head, slowly

Every piece of the equation, and why it is there.

The goal, made concrete



3Blue1Brown

Sentence: *"a fluffy blue creature roamed the verdant forest."*

We want the noun **creature** to absorb its adjectives **fluffy** and **blue**, so its vector means *"a fluffy, blue creature."*

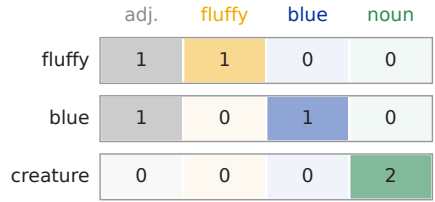
Two questions the mechanism must answer:

- Which words are **relevant** to creature?
- **What** should they hand over?

Predict first: who does “creature” listen to?

Toy vectors with 4 numbers, read as [adjective, fluffy, blue, noun]. A darker cell is a bigger number. (A real embedding has no named cells: “noun” is a direction spread over all 768 numbers, LLM-3. The toy names them so we can read along.)

No learned matrices yet: each vector is its own query, its own key and its own value, the simplified version from LLM-3. Score = dot product divided by $\sqrt{4} = 2$; softmax by rows; weighted sum.



Predict: which of the three words gets the biggest share of creature’s attention?

Itself

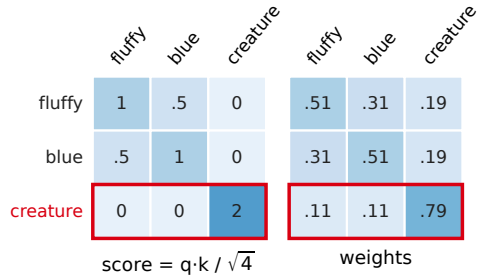
creature · creature $= 4$, so its own score is $4/2 = 2$; against fluffy and blue it scores 0.

Softmax: $(1, 1, e^2)/(2 + e^2) = (1, 1, 7.389)/9.389 = (0.107,\ 0.107,\ \mathbf{0.787})$: creature spends **79%** of its attention on itself. Its update:

$$0.107\,(1, 1, 0, 0) + 0.107\,(1, 0, 1, 0)$$

$$+\,0.787\,(0, 0, 0, 2) = (0.21,\ 0.11,\ 0.11,\ 1.57)$$

Mostly more creature. And the **scores** are symmetric: fluffy · blue = blue · fluffy (softmax runs row by row, so the weights can still differ).



Raw vectors ask "who is like me?", and the answer is always "me". Creature wants its adjectives.

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Piece 1: $QK^\top$

Who matches whom? Creature needs a better question than "who is like me?".

A soft dictionary lookup

hard lookup: a Python dict

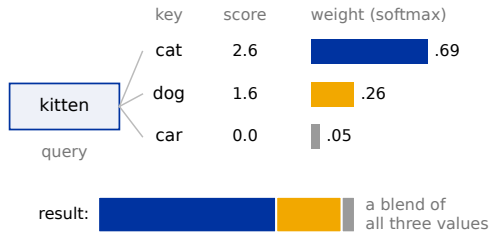
```
d = {"cat": v1, "dog": v2, "car": v3}

d["cat"]          -> v1

d["kitten"]       -> KeyError
```

one key matches exactly,
or nothing does

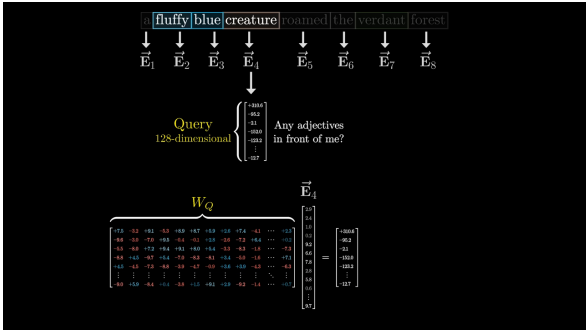
soft lookup: attention



A **query** is what you want. A **key** is the label an entry is filed under. A **value** is what the entry contains.

A dictionary is a **hard** lookup: one key matches exactly, or nothing does. Attention is a **soft** lookup: *every* key matches a little, and you get a blend of all the values. Every word makes its own query, key and value, each with its own learned matrix. That is how creature can stop asking "who is like me?".

Query: "what am I looking for?"



Multiply a word's vector by a learned matrix W_Q ($d_k \times d$, so 64×768; the **same** W_Q for every word) to get its **query**, a shorter vector; its length is called d_k (64 per head in GPT-2):

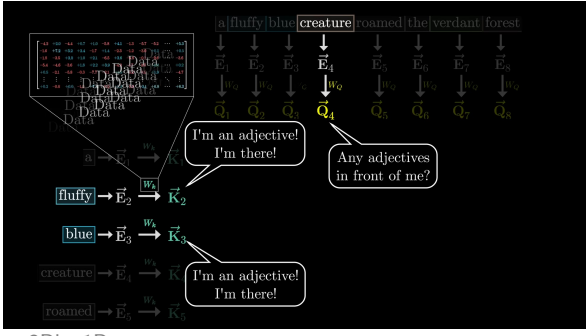
$$\mathbf{q}_i = W_Q\,\mathbf{e}_i.$$

Creature's query can encode the question *"any adjectives in front of me?"*

W_Q is **learned**: nobody writes "adjectives?" into it. Backprop from the next-token loss adjusts it like any other weight; the questions that survive are the ones that help predict.

3Blue1Brown (drawn for GPT-3: 128 numbers per query; GPT-2 uses 64)

Key: "what do I offer?"



3Blue1Brown

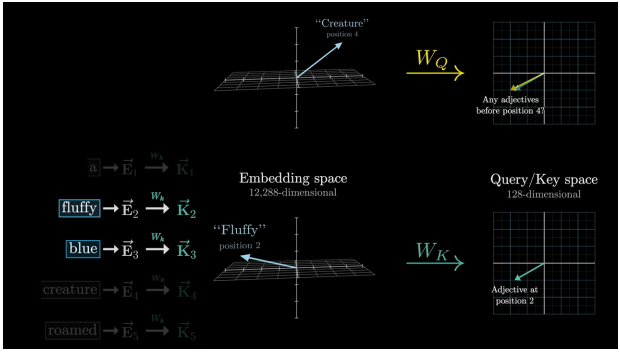
A second learned matrix W_K turns each vector into a **key**:

$$\mathbf{k}_j = W_K\,\mathbf{e}_j.$$

A key *answers* queries. Fluffy's key can encode *"I am an adjective, sitting right here."*

A key is a word *advertising* itself, so that the right queries can find it.

Queries and keys meet in one space



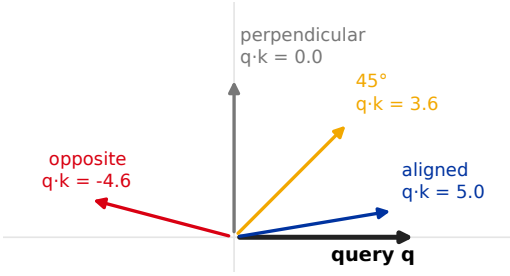
W_Q and W_K **project** the big embedding space down to a shared, smaller **query/key space** (project: multiply by a matrix that makes the vector shorter, 768 numbers in, 64 out).

Good W_Q, W_K arrange it so that creature's query ("adjectives before me?") and fluffy's key ("adjective, here") point the **same way**.

That is all W_Q and W_K do: learn a space where "relevant" means "pointing the same way".

3Blue1Brown

The dot product scores relevance



$\mathbf{q} \cdot \mathbf{k} = |\mathbf{q}|\,|\mathbf{k}| \cos\theta$ is **large** when the two point the same way, **zero** when they are perpendicular, and **negative** when they point apart.

So "how relevant is this key to my query?" collapses to **one number**.

All queries against all keys at once is one matrix product: $S = QK^\top$. Row i holds word i's scores against every word.

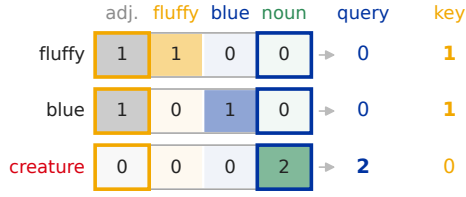
By hand: ask a different question

Make the query/key space as small as it gets: **one** number, a single axis that means "adjective match". (GPT-2's heads use 64 numbers: room for 64 kinds of question.)

$w_Q = (0, 0, 0, 1)$ reads the noun cell: a **noun** asks along that axis, "any adjectives?".

$w_K = (1, 0, 0, 0)$ reads the adjective cell: an **adjective** answers on the same axis, "adjective here".

Score = query × key, divided by $\sqrt{1} = 1$; then softmax by rows, as before.



blue box: w_Q reads it. orange box: w_K reads it.

Predict: what are creature's three weights now? And whom does fluffy listen to?

Creature finds its adjectives

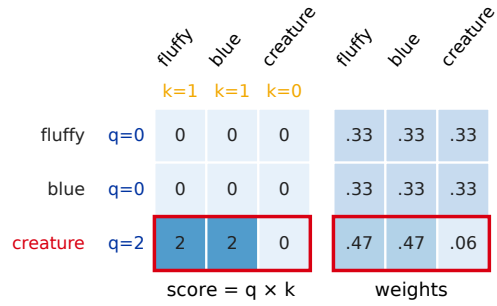
Creature's scores:
$2\times 1,\ 2\times 1,\ 2\times 0 = 2,\ 2,\ 0$. Softmax:

$$\frac{(e^2,\ e^2,\ e^0)}{2e^2+1} = \frac{(7.39,\ 7.39,\ 1)}{15.78}$$

$$= (\mathbf{0.47},\ \mathbf{0.47},\ 0.06)$$

Fluffy's query is 0, so all its scores are 0: it asks for nothing and spreads evenly, 0.33 each.

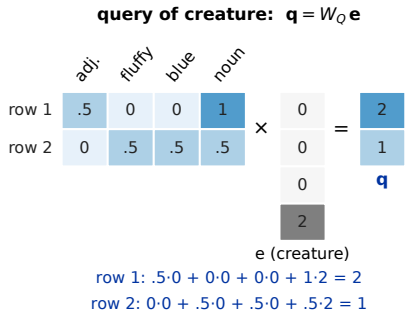
Each **row** sums to 1, a word's attention budget; the columns do not.



Creature now asks "any adjectives?" (79% on itself before, 6% now). And the grid is no longer symmetric: creature → fluffy scores 2, fluffy → creature scores 0.

By hand: a real W_Q is a matrix

A real query has d_k numbers (64 in GPT-2), so W_Q is a $d_k \times d$ matrix. The same three words with $d_k = 2$:



Each **row** of W_Q is one question, a direction; each number of **q** is how far **e** points along it, a dot product. The rest the same way: keys blue $(1,\ 0.5)$, creature $(0,\ -1)$; queries fluffy and blue $(0.5,\ 0.5)$. Made-up numbers again, but no single cell is "the noun question" any more: row 1 mixes the adjective and noun cells, as a learned matrix does.

By hand: scores from 2-number queries and keys

		fluffy	blue	creature
	k =	(1, .5)	(1, .5)	(0, −1)
fluffy	q = (.5, .5)	0.53	0.53	−0.35
blue	q = (.5, .5)	0.53	0.53	−0.35
creature	q = (2, 1)	1.77	1.77	−0.71

score = q·k / $\sqrt{2}$

	fluffy	blue	creature
fluffy	.41	.41	.17
blue	.41	.41	.17
creature	.48	.48	.04

weights (softmax by rows)

Creature · fluffy: $2 \times 1 + 1 \times 0.5 = 2.5$, divided by $\sqrt{2}$: 1.77. Creature · itself: $2 \times 0 + 1 \times (-1) = -1$, divided by $\sqrt{2}$: -0.71. Softmax of $(1.77,\ 1.77,\ -0.71)$: $(0.48,\ 0.48,\ 0.04)$.

The same story as the one-number toy's $(0.47,\ 0.47,\ 0.06)$, now from real matrix products. With 64 numbers per query it is the same arithmetic with longer sums.

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Piece 2: softmax

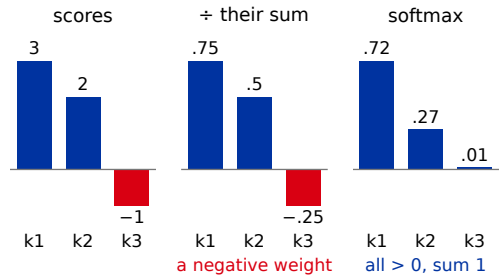
Scores can be any number, even negative. Weights must be positive and add up to 1.

Why softmax? 1: positive weights that sum to 1

Raw scores can be any real number. But we want a **weighted average** of the values, so the weights must be

- non-negative, and
- sum to 1 (a distribution over "where to look").

Dividing by the sum fixes the second but not the first (middle): a negative score gives a negative weight. **Softmax** exponentiates first, so every weight is positive, then divides.



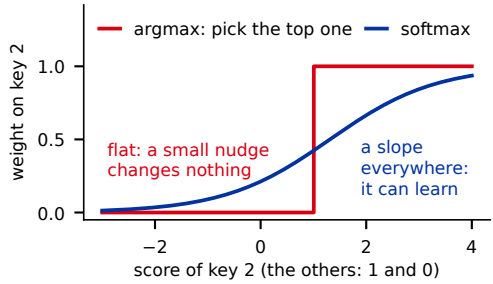
$\mathrm{softmax}(s)_j = e^{s_j}/\sum_i e^{s_i}$: positive because $e^x > 0$ for every x, and the division makes the weights sum to 1.

Why softmax? 2: soft, so it can learn

Why not just pick the best key (argmax)? Slide one key's score and watch its weight.

argmax jumps from 0 to 1 the moment the key takes the lead, and is flat everywhere else. A small change of the score changes nothing, so the gradient is 0 and backprop gets no signal.

softmax moves a little for every little change: a slope everywhere, so training can push the weights where they need to go. And it spreads weight over all the good keys instead of picking one.



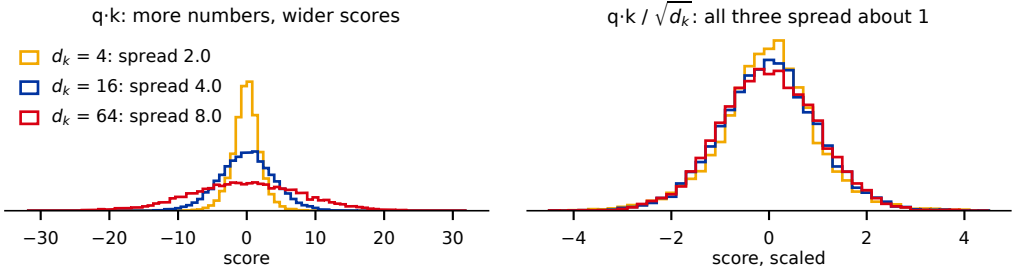
One catch: the weights must sum to 1, so a head can never look at *nothing*. LLM-5 shows where real heads park the weight they do not need (attention sinks).

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Piece 3: $\sqrt{d_k}$

Why divide by this one number? **Predict:** do scores from 64-number vectors come out bigger or smaller than from 4-number ones?

More numbers, bigger scores



A dot product over d_k numbers is a sum of d_k products. If the numbers are independent, with mean 0 and spread 1 (about right at the start of training), each product has variance 1, and variances add: the sum has variance d_k, spread $\sqrt{d_k}$. Measured on 10,000 random pairs each: about 2, 4 and 8 (8.05 for $d_k = 64$).

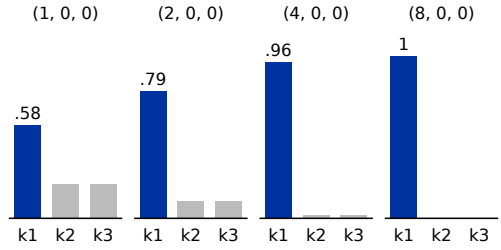
Bigger. GPT-2's heads use $d_k = 64$, so raw scores swing by about ± 8, not ± 1. Right: divide by $\sqrt{d_k}$ and all three are back to spread 1.

Big scores freeze softmax

The same three keys, with the winner's lead growing: $(1,0,0)$, $(2,0,0)$, $(4,0,0)$, $(8,0,0)$. Softmax goes from a soft preference to one weight near 1: it **saturates**.

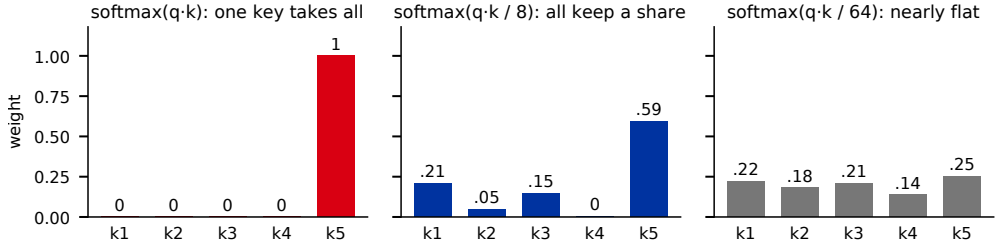
Now nudge a loser up by 1. At $(8,0,0)$ its weight goes from 0.0003 to 0.0009. At $(1,0,0)$ the same nudge moves it from 0.21 to 0.42.

The slope of a weight with respect to its own score is $w(1-w)$: for that loser 0.0003 at $(8,0,0)$, 0.17 at $(1,0,0)$. A slope near 0 gives the gradient nothing to work with: **training stalls**, argmax's flat part all over again.



The fix: divide by $\sqrt{d_k}$

Divide every score by the spread, $\sqrt{d_k} = 8$: back to about ± 1, softmax stays soft, every key keeps a slope. Five random keys, $d_k = 64$, the same scores three ways:



Why not divide by $d_k = 64$ (right)? It squashes the scores near 0 and softmax goes flat: every key gets about the same weight, however relevant. $\sqrt{d_k}$ is the number that makes the spread exactly 1.

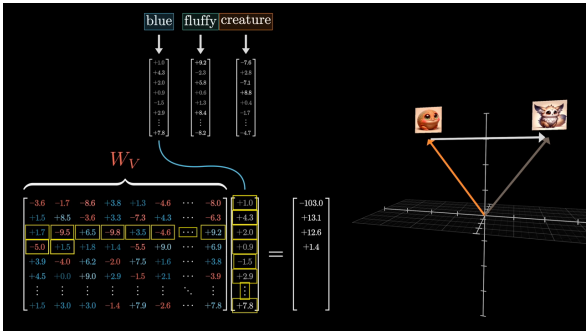
The same in code: Karpathy, "Let's build GPT", 1:16:56 (first slide).

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Piece 4: V

The weights say whom to listen to. What do those words actually say?

Value: "what do I hand over?"



3Blue1Brown

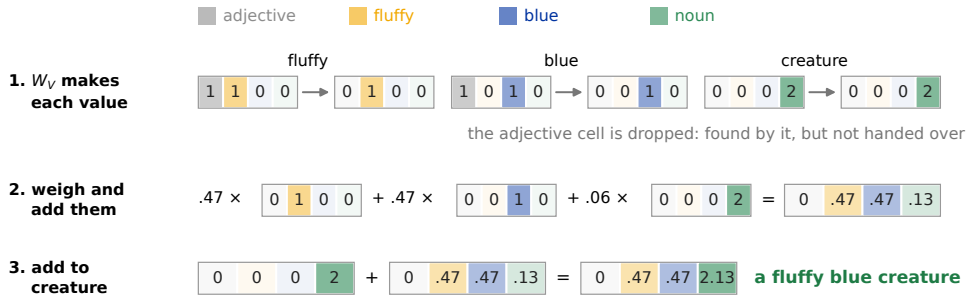
A third learned matrix W_V makes each word's **value**, the information it hands over when attended to:

$$\mathbf{v}_j = W_V\,\mathbf{e}_j.$$

Fluffy's value is, roughly, "the fluffiness to add to a noun".

Key = how you are *found*; value = what you *give*. Different jobs, different matrices.

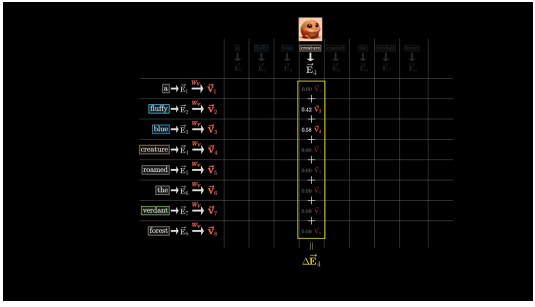
By hand: what gets handed over



Creature's weights 0.47, 0.47, 0.06 come from piece 1 (0.468 and 0.063 to three digits, so $0.063 \times 2 = 0.13$). Step 3 is the residual connection (LLM-5). With raw vectors the update was (0.21, 0.11, 0.11, 1.57), so creature became (0.21, 0.11, 0.11, 3.57): mostly more creature.

Fluffy was *found* by its label ("adjective") and *handed over* its content (fluffiness). Found by one thing, give another: that is why $W_K \neq W_V$.

The output: values, weighted and summed



3Blue1Brown

In general: the weights say *how much* to listen, the values say *what* to pass. Word i's output is

$$\mathbf{o}_i = \sum_j \alpha_{ij}\,\mathbf{v}_j$$

(α_{ij}: the weight word i gives word j.)

Every word is updated this way **at once**, in one parallel step. The output is **added** to the word's own vector, so words *refine* rather than overwrite (the residual connection; LLM-5's W^O first maps a head's 64 numbers back to 768).

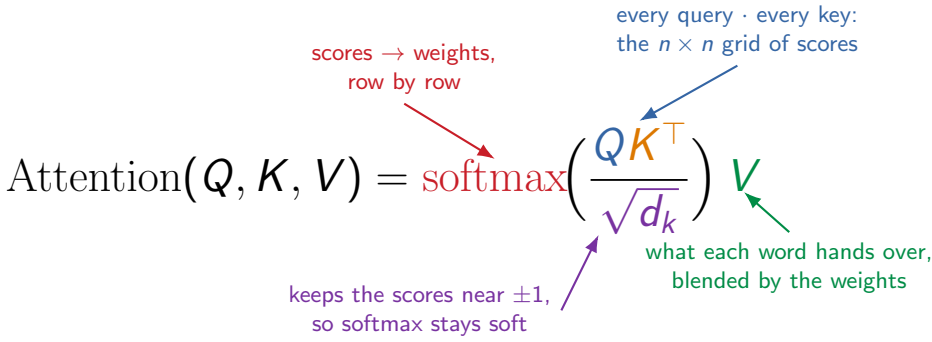
Why three matrices?

One word plays **three roles** at once:

Query $\mathbf{q} = W_Q\mathbf{e}$	what I am *looking for*	creature: "any adjectives?"
Key $\mathbf{k} = W_K\mathbf{e}$	how I am *found*	fluffy: "adjective, here"
Value $\mathbf{v} = W_V\mathbf{e}$	what I *hand over*	fluffy: fluffiness, not the label

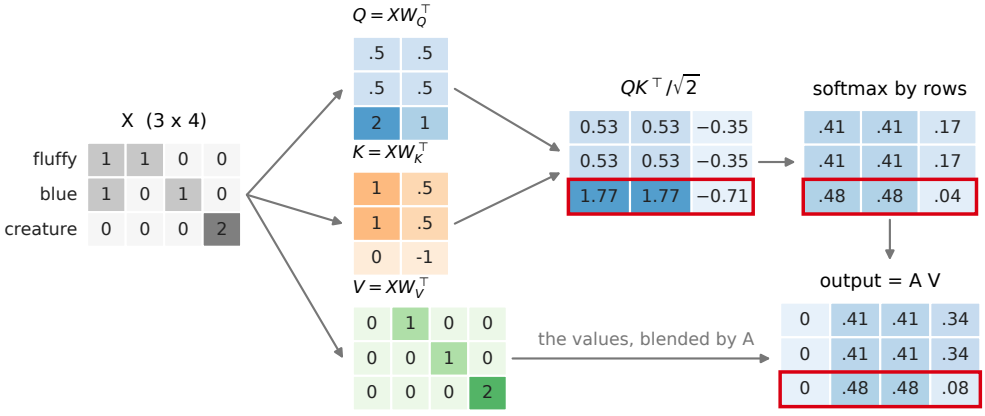
Without them, every word asks "who is like me?" and finds itself (79% in our toy), and A and B score each other exactly the same. One shared matrix for queries and keys would do that too: $W_Q = W_K$ gives $\mathbf{q}_A \cdot \mathbf{k}_B = \mathbf{q}_B \cdot \mathbf{k}_A$. With them, the question, the label and the message are three separate things the model learns. They are the only weights in a single head (LLM-5 adds W^O when heads are combined).

Putting it together: every piece has a job



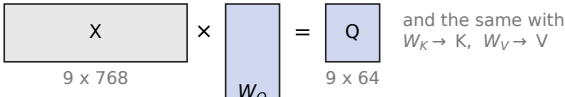
"Score, softmax, weighted sum": the same three moves as wall 1, now with learned Q, K, V from one sentence. In wall 1 the queries came from the decoder and the keys and values from the encoder: that version is **cross-attention** (LLM-6).

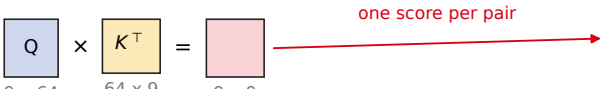
By hand: the whole line as matrices



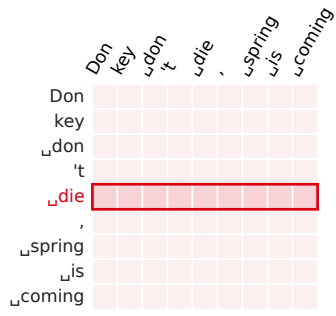
The matrix toy from piece 1, one token per row of X: three matrix products, the score grid, softmax by rows, then AV. Creature's row (red) ends at (0, 0.48, 0.48, 0.08): a fluffy, blue creature, as in piece 4. Every row is computed at once.

Shapes, on a real sentence





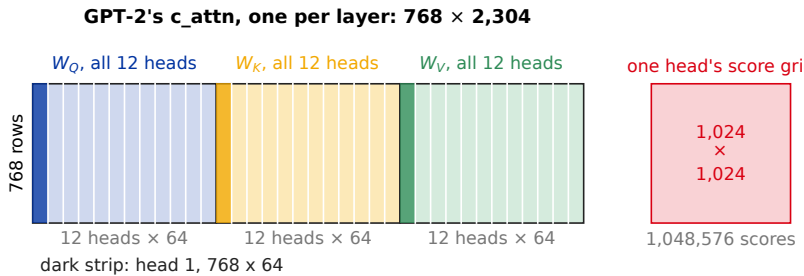
softmax each row, then multiply by V (9 x 64):
the output is 9 x 64. Boxes not to scale.



row = who asks, column = who answers;
the red row: what ␣die looks at

"Donkey don't die, spring is coming" is 6 words but 9 GPT-2 tokens (LLM-1): n counts tokens. The sizes 768 and 64 are GPT-2's.

The same shapes in GPT-2



One head's W_Q, W_K, W_V are 64×768 each in our $W_Q\,\mathbf{e}$ notation; the file stores them **transposed**, all twelve heads' side by side, because code multiplies rows (next slide). Why 64 and not 768? $12 \times 64 = 768$: each head gets its own small space for its own question (LLM-5).

A full 1,024-token window: 150,994,944 scores over 12 heads and 12 layers. Double the text and the grid grows **four times**; X, Q, K, V only double (LLM-6 adds up the bill). (GPT-2's config and weight file, 2026-10-05.)

Attention in about 10 lines

$\mathrm{softmax}(QK^\top/\sqrt{d_k})\,V$

Every step is a matrix operation. In NumPy, with one token per **row** of `X`, so $\mathbf{q} = W_Q\,\mathbf{e}$ becomes `X @ Wq` with `Wq` $= W_Q^\top$ (768×64):

```
import numpy as np

def attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv        # three learned views
    d_k = Q.shape[1]
    scores = Q @ K.T / np.sqrt(d_k)          # relevance, scaled
    scores -= scores.max(1, keepdims=True)   # safe exp, same softmax
    weights = np.exp(scores)
    weights /= weights.sum(1, keepdims=True)  # softmax over each row
    return weights @ V                       # weighted sum of values
```

No loops over positions, no recurrence: three matrix products and a softmax. That simplicity is the whole point. Subtracting each row's max changes no weight (softmax only sees differences, LLM-6) and keeps exp from overflowing.

Run the 10 lines on the toy

The matrix toy, fed to the function on the previous slide:

```
X   = np.array([[1, 1, 0, 0],                    # fluffy
                [1, 0, 1, 0],                    # blue
                [0, 0, 0, 2]])                   # creature
W_Q = np.array([[0.5, 0, 0, 1], [0, 0.5, 0.5, 0.5]])
W_K = np.array([[1, 0, 0, 0], [0, 0.5, 0.5, -0.5]])
W_V = np.diag([0, 1, 1, 1])
out = attention(X, W_Q.T, W_K.T, W_V.T)          # tokens are rows: Wq = W_Q^T
print(np.round(out, 2))
```

```
[[0.   0.41 0.41 0.34]
 [0.   0.41 0.41 0.34]
 [0.   0.48 0.48 0.08]]
```

The last row is creature's result from "the whole line as matrices", to the digit. Fluffy and blue ask the same question (the same **q**), so they get the same weights and the same row.

Masking

Who may look at whom? Two ways to learn from raw text.

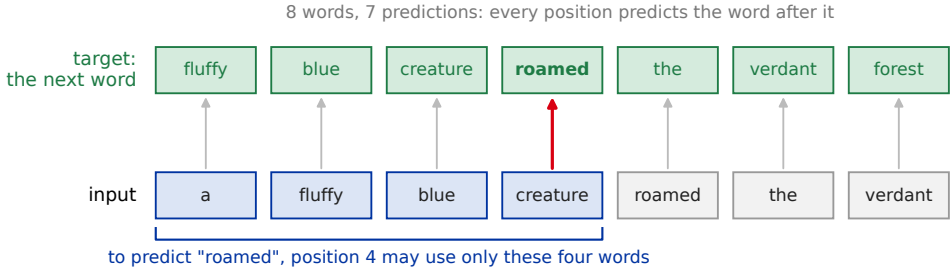
Two fill-in games

A model learns from raw text by playing a fill-in game: hide a word, guess it, compare with the truth. No labels needed. Two versions on our sentence:



Predict: which words may the model use to fill each blank? And which game teaches a model to *write*, one word after another?

Game 1: causal language modeling



Shift the sentence by one: the target at every position is the next word. Each position may use only the words up to itself: the past can cause the next word, the future cannot. Hence the name **causal language modeling** (CLM), also called next-token prediction or *autoregressive* modeling.

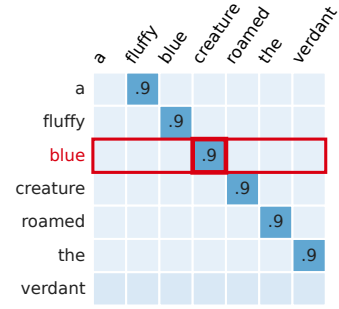
This is GPT's game (Radford et al., 2018), and what GPT-2 was trained on. Writing is the same game for real: predict a word, append it, predict again (LLM-6, LLM-2).

The cheat

Self-attention lets every word look at every word. Train game 1 like that, and "blue", which must predict "creature", can simply **look at** "creature": it is the next word in the input.

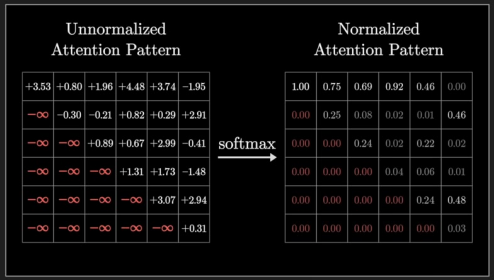
One head learns "copy the word to my right" (it can tell right from left because the position vectors went in before the first layer, LLM-3), and the training loss falls towards 0.

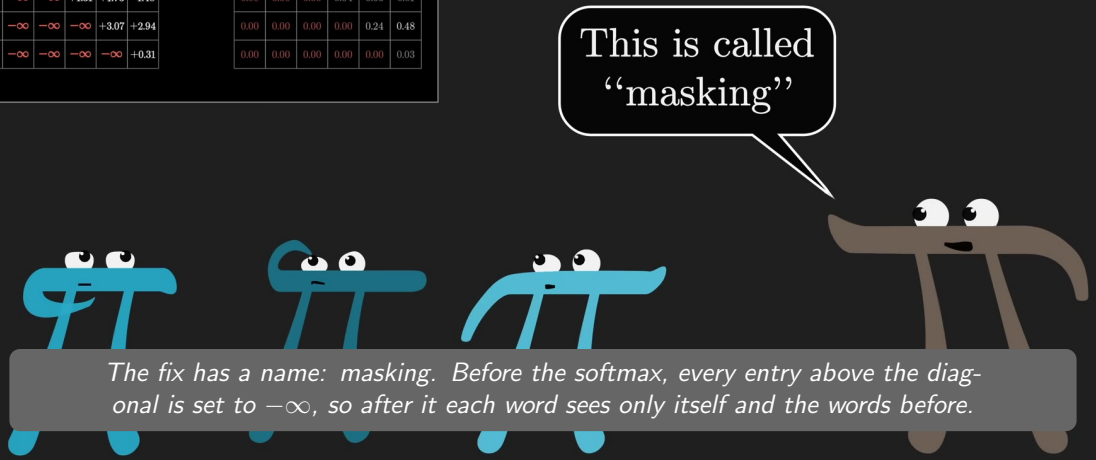
Then you ask it to write. The next word does not exist yet, there is nothing to copy, and the model has learned nothing it can use.



no mask: a head that copies the next word.
"blue" must predict "creature": it is right there

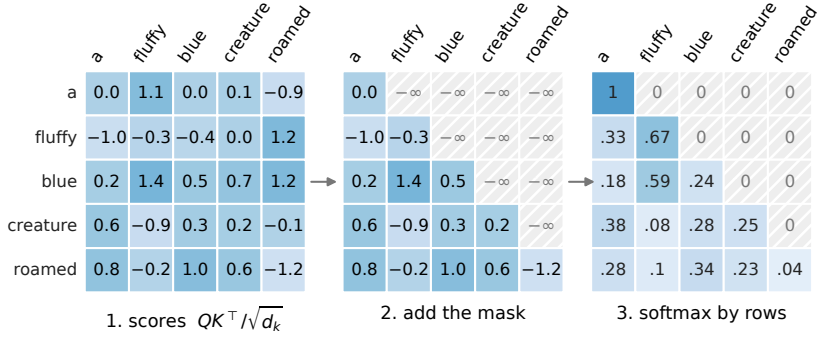
Whatever a position looks at in training must also exist when the model writes. So: forbid every position to look to its right.





The fix has a name: masking. Before the softmax, every entry above the diagonal is set to $-\infty$, so after it each word sees only itself and the words before.

The mask in three steps



Add a matrix M to the scores *before* the softmax: 0 on and below the diagonal, $-\infty$ above it. $e^{-\infty} = 0$, so every future word gets weight **exactly** 0, and each row still sums to 1.

$\mathrm{softmax}\big(QK^\top/\sqrt{d_k} + M\big)\,V$: the same line plus one fixed matrix, in every attention layer of GPT. No new weights, nothing to learn.

By hand: the mask on creature's grid

The learned grid from piece 1 (queries ask "any adjectives?", keys answer "adjective here"), now with the mask:

- **fluffy** (red) sees only itself: $(0.33, 0.33, 0.33) \to (1, 0, 0)$;
- blue sees fluffy and itself, scores $0, 0$: $(0.5, 0.5, 0)$;
- creature's row does not change: $(0.47, 0.47, 0.06)$.



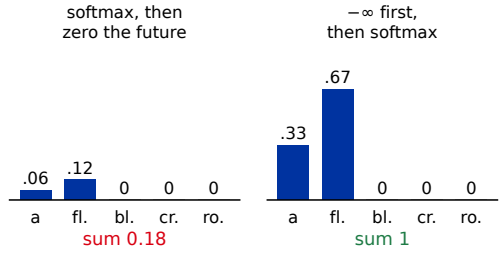
In English adjectives come before their noun, so a left-to-right model can still let the noun gather them. What the mask removes is the other direction.

Why $-\infty$ before the softmax, and not 0 after?

Row "fluffy" of the three-step grid: scores -1.0 and -0.3 for a and fluffy, then -0.4, 0.0 and 1.2 for the three words after it.

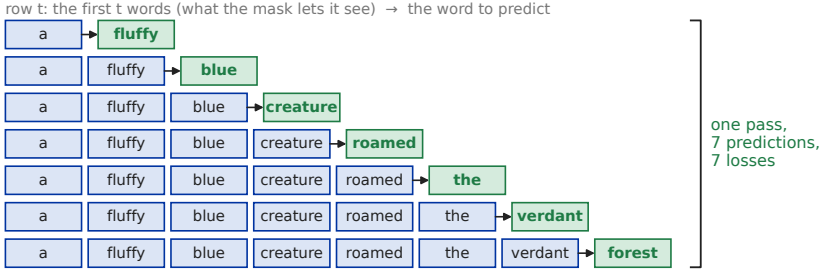
Zero after: softmax over all five hands the future 82% of the weight ("roamed" alone 0.54). Zero those and what is left, $0.06 + 0.12$, sums to 0.18: no longer a weighted average.

$-\infty$ **before:** the softmax only ever sees a and fluffy: 0.33 and 0.67. The same 1 to 2 ratio, now summing to 1.



Masking is a softmax over the visible keys only. (Zeroing after and dividing by 0.18 gives the same answer; adding $-\infty$ does it in one step.)

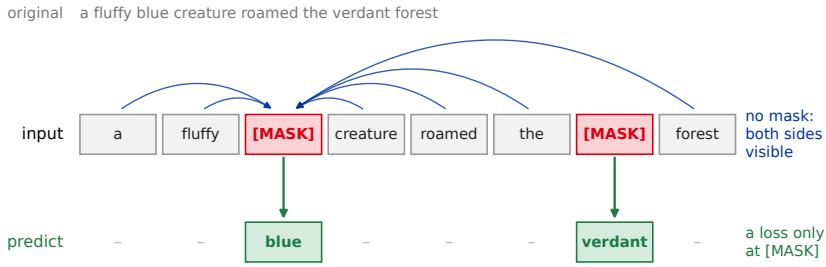
One pass, every prefix at once



Without the mask, each prediction needs its own pass with the future cut off: feed the first t words, predict word $t{+}1$, seven times. With the mask, row t of the grid *is* that cut: one pass over 8 words gives **7 training signals**, none can cheat.

A large part of why pretraining on internet-scale text is affordable at all. The causal mask is not a safety measure; it is a **throughput** decision that happens to enforce correctness (LLM-7 comes back to it).

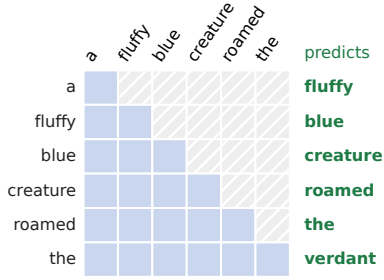
Game 2: masked language modeling



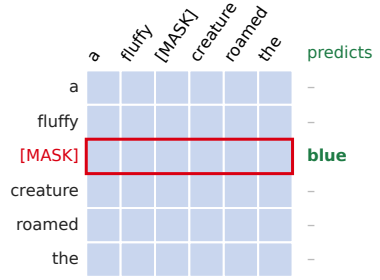
Replace some words with a special token, `[MASK]`, and predict the originals. **No attention mask**: every word, `[MASK]` included, sees both sides. This is **masked language modeling** (MLM), the game of BERT, Google's model for reading rather than writing (Devlin et al., 2019; LLM-6): about 15% of the tokens are hidden (here 2 of 8), and only those give a loss.

The predict's answer: game 1 uses only the words before the blank, game 2 both sides, and only game 1 can write. BERT's exact recipe is in LLM-7.

CLM vs MLM



CLM (GPT): the causal mask



MLM (BERT): no attention mask

	CLM (GPT)	**MLM** (BERT)
who may look	the past only	everyone, both sides
a loss at	every position: n per pass	the hidden ones: about $0.15\,n$
can it write?	yes, one word at a time	no: it needs the words on both sides

Two different things called "mask"

the [MASK] token

hides a word in the input



MLM (BERT): the word is gone, everything else is visible

the attention mask

hides positions from each other

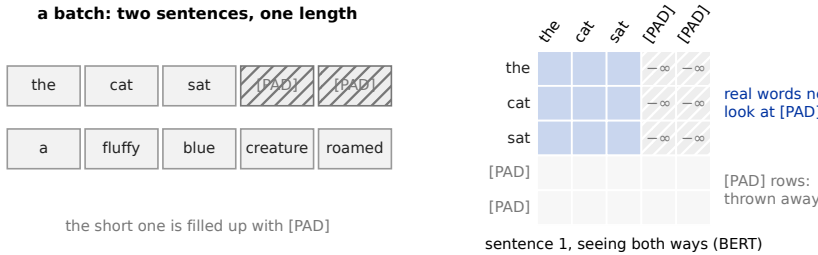


no word is removed

CLM (GPT): the input is complete, each row sees only the past

`[MASK]` is a **token**: it changes the input, and the model sees everything else (MLM, BERT). The **attention mask** is a matrix added to the scores: the input is complete, but each position may look only at some of the others (CLM, GPT). One word, two different mechanisms.

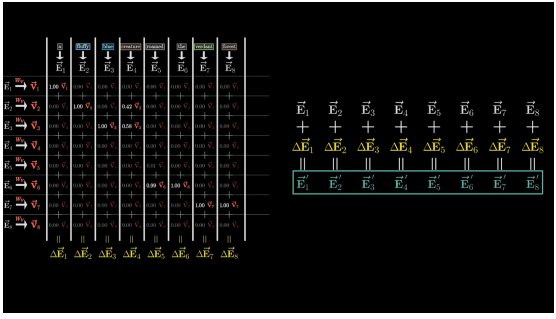
One more mask: padding



A GPU wants a batch to be one rectangle, so shorter sentences are filled up with a `[PAD]` token. Nobody should read it: its columns get $-\infty$ too (the same trick), and the outputs at the `[PAD]` rows are thrown away.

With a causal mask and the padding at the end, real words never see it anyway. It matters when the model sees both ways (BERT), and when prompts are padded at the *front* for writing, so every prompt ends in the same column, where the next word goes. Pretraining packs documents end to end and needs no padding (LLM-7).

One head, end to end



3Blue1Brown

- **Wall 1**, one C: every step gets its own weighted mix of the inputs.
- **Wall 2**, the queue: self-attention builds every vector at once.
- Each word makes a query, a key and a value: ask, get found, hand over. Without them a word listens to itself.
- Score $QK^\top/\sqrt{d_k}$, softmax by rows, weighted sum of values; the grid is $n \times n$.
- The **causal mask** keeps CLM (GPT) from peeking; MLM (BERT) hides words and sees both sides.

Next (LLM-5): many heads at once, and the block around them.

Try it yourself

Back to the demo from the first slide: **Transformer Explainer** (`poloclub.github.io/transformer-explainer`) runs GPT-2 small in your browser. Type your own sentence and look at the attention weights: the same $QK^\top$, softmax and $\times V$ as today, on a real model.

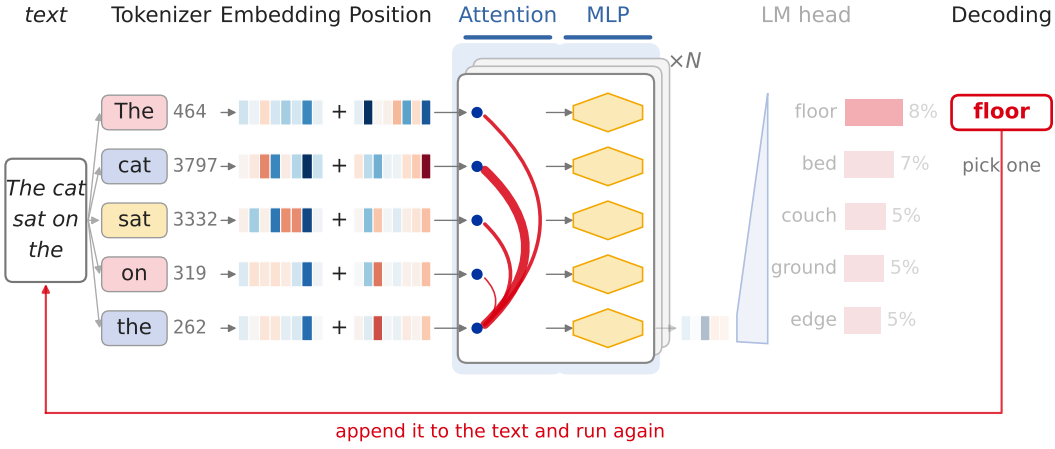
Real heads are messier than the creature toy, and different heads do different jobs. That is where LLM-5 starts.

Careful: attention weights are not explanations

- The grids are tempting: "the model looked at fluffy, so that is why it answered". Jain & Wallace (NAACL 2019) tested it: attention weights often disagree with other measures of which inputs matter, and very different weights can give the same prediction. Their title: "Attention is not Explanation".
- Wiegreffe & Pinter (EMNLP 2019) answered with "Attention is not *not* Explanation": it depends on what you call an explanation, and the swapped-in weights fail closer tests.
- Both studied RNNs with attention, not GPT. The lesson carries over: a weight shows where one head looked, not why the model answered.

Use the demo to see *what* heads do. Testing *why* takes interventions: change something inside the model and watch the answer (LLM-5 has one; the interpretability chapter has many).

Next on the map



Next (LLM-5): many heads at once, and the block around them.

Sources & further reading

- Bahdanau, Cho & Bengio (2015), *Neural Machine Translation by Jointly Learning to Align and Translate*: attention, for translation.
- Xu et al. (2015), *Show, Attend and Tell*: attention for image captioning.
- Vaswani et al. (2017), *Attention Is All You Need*: self-attention, the Transformer.
- Radford et al. (2018), *Improving Language Understanding by Generative Pre-Training* (GPT, causal LM); Devlin et al. (2019), *BERT* (masked LM), NAACL.
- Jay Alammar, *The Illustrated Transformer*; Andrej Karpathy (2023), Let's build GPT: from scratch, in code, spelled out.
- Jain & Wallace (2019), "Attention is not Explanation", NAACL; Wiegreffe & Pinter (2019), "Attention is not not Explanation", EMNLP. Levy (2024), Wired, on the paper's history.
- Transformer Explainer: GPT-2 small in the browser, `poloclub.github.io/transformer-explainer`.
- **3Blue1Brown**, "Deep Learning" chapters 5 and 6: the visual intuition this deck leans on.

For the full, animated story of attention, watch 3Blue1Brown chapter 6. It is the clearest explanation out there.